Desenho da arquitetura

Revenda de veículos · Tech Challenge SOAT · Fase 5 · 05/11/2026

> Esta entrega descreve código e infraestrutura declarada. `infra/config/dev.yml` contém `destroy: true`: o ambiente foi implantado e validado em 05/11/2026 e depois removido, e não há implantação ativa. Testes locais não comprovam latência, custos, achados de segurança ou comportamento E2E na AWS.

1. Objetivo e atendimento ao enunciado

API de revenda de veículos: cadastro e edição do catálogo, cadastro de compradores, listagens por preço crescente, reserva concorrente, código de pagamento, pagamento, desistência, retirada e baixa. O frontend React é uma entrega adicional. O pagamento é uma simulação Pix, sem integração bancária real; a equipe aprova ou recusa o pagamento pelo painel.

Código: `github.com/Edufreitass/fase5-tech-challenge-soat` (se privado, o avaliador deve receber acesso). PDFs em `docs/deliverables/`; fontes dos PDFs em `docs/deliverables/src/`; diagramas editáveis em `docs/diagrams/`.

2. Serviços e justificativas (serverless e gerenciados)

Serviço	Uso implementado	Razão da escolha
API Gateway REST	Contrato OpenAPI, integração com Lambdas, authorizer Cognito e throttling	Interface gerenciada para o frontend e o Swagger; sem servidor a operar
Lambda (Python 3.13, arm64)	Handlers REST e workers, uma role IAM por função	Paga por uso; arm64 reduz custo; escala sem administração
DynamoDB	Seis tabelas: veículos, clientes, vendas, pagamentos, idempotência e gateway simulado	Escritas condicionais e transações resolvem a concorrência; o GSI com preço na sort key entrega a listagem ascendente
Step Functions Standard	SAGA orquestrada com esperas por task token, timeouts e compensações	Esperas de dias, compensações explícitas e histórico por execução
Cognito	Dois user pools (clientes e funcionários), MFA obrigatório na equipe	Autenticação gerenciada; separa populações de risco distinto
SNS e SQS	Eventos de domínio, fila de auditoria e fila de webhooks do gateway simulado, com DLQ (14 dias)	Desacopla o que não deve bloquear o fluxo principal
DynamoDB Streams	Detecta CRIADO → RESERVADO e inicia a SAGA	O fluxo nasce do dado persistido, sem publicar em dois lugares
EventBridge Scheduler	Varredura de vendas órfãs e rotina diária de retenção LGPD	Agendamento gerenciado, sem servidor de cron
SSM Parameter Store (SecureString)	Segredos e chave privada de criptografia por campo	Segredos fora do código, cifrados com a chave KMS gerenciada pela AWS
CloudWatch e X-Ray	Logs, métricas, 9 alarmes, painel e tracing	Observabilidade nativa, sem agentes

S3 privado + CloudFront	SPA estática com OAC e cabeçalhos de segurança; encaminha /api/* ao API Gateway	Site estático sem servidor; não há VPC provisionada
Terraform + GitHub Actions (OIDC)	Infraestrutura declarativa e credenciais temporárias no pipeline	Ambiente reproduzível; sem chaves de longa duração

3. Consistência e fluxo de compra

A API valida o comprador e grava a venda CRIADO. Em uma segunda transação, reserva o veículo disponível, cria o lock do cliente, muda a venda para RESERVADO e grava o histórico. Uma disputa perdedora recebe 409 e sua venda é cancelada. O Stream observa a transição e inicia uma execução nomeada por vendaId, evitando duplicação.

A SAGA emite a cobrança, espera o pagamento, prepara a retirada e conclui a venda com o veículo VENDIDO. Desistência, expiração e ausência de retirada têm compensações (detalhes no relatório SAGA).

4. Serviços de segurança utilizados

Serviço	Uso	Por que
Cognito	Autenticação em dois pools; MFA obrigatório para funcionários	A aplicação não guarda senhas; o MFA reduz o risco de conta de funcionário comprometida
API Gateway (authorizer e throttling)	Valida o token antes da Lambda e limita a taxa	Rejeita na borda o que não está autenticado e reduz abuso
IAM (role por Lambda + permissions boundary)	Menor privilégio; a chave privada PII é legível em runtime só por clientes-interno e lgpd-worker	Limita o dano se uma função for comprometida
SSM Parameter Store + KMS gerenciado	Segredos e chave privada cifrados em repouso	Segredos fora do código e do repositório
S3 privado + CloudFront (OAC, cabeçalhos)	Hospedagem do frontend	Sem acesso público ao bucket; todo acesso passa pelo CloudFront
HMAC e SigV4	Webhook público valida assinatura e timestamp; rotas internas do gateway simulado usam IAM/SigV4	Impede webhook falso ou repetido
CloudWatch Logs, SNS e SQS	Trilha de auditoria (retenção de 1827 dias) e logs de aplicação (7 dias)	Permite saber quem acessou dados pessoais
OIDC do GitHub Actions	Credenciais temporárias no pipeline	Sem chaves AWS de longa duração

Controles de segurança não implementados (apenas documentados)

Os controles abaixo **não estão implementados** no código nem no Terraform: foram deixados de fora por custo, pois o projeto é acadêmico e roda em conta pessoal. Estão apenas documentados como evolução para produção. Os valores variam por região e mudam com o tempo; confirme nas páginas de preço da AWS.

Controle	O que protegeria	Justificativa de custo	Mitigação atual
AWS WAF	Bloqueio de ataques web na borda	Mensalidade fixa por Web ACL e por regra, mais cobrança por requisição; sem cota gratuita	Throttling no API Gateway, validação de entrada, Cognito, cabeçalhos de segurança
Amazon GuardDuty	Detecção gerenciada de ameaças na conta	Gratuito só durante a avaliação; depois cobra por volume de eventos analisados	Alarmes CloudWatch e auditoria de aplicação
AWS Security Hub	Consolidação de achados e padrões (CIS, FSBP)	Gratuito só na avaliação; depois cobra por verificação e achado, e depende do AWS Config	Revisão de IaC, bandit, pip-audit, gitleaks e zizmor no CI
Amazon Macie	Descoberta de dados sensíveis em S3	Cobra por bucket e por dado inspecionado; dados pessoais não ficam em S3 (só o site estático)	Dados pessoais no DynamoDB, com CPF/RG/CNH cifrados por campo
Chave KMS gerenciada pelo cliente	Política, rotação e auditoria próprias da chave	Mensalidade por chave e cobrança por uso; os segredos usam `alias/aws/ssm` (chave gerenciada pela AWS)	Cifra por campo (libsodium) com chave privada em SSM SecureString, lida só por duas funções
DynamoDB PITR	Restauração a um ponto no tempo	Cobrado por GB-mês de backup contínuo, fora do free tier	Tabelas e dados iniciais recriáveis por Terraform e migrações; vendas reais não são recuperáveis
CloudTrail próprio	Auditoria permanente das chamadas à API AWS	Trail adicional exige armazenamento em S3 e, para eventos de dados, cobrança por evento	Histórico padrão da conta (90 dias) e auditoria de aplicação (SNS → SQS → CloudWatch Logs)

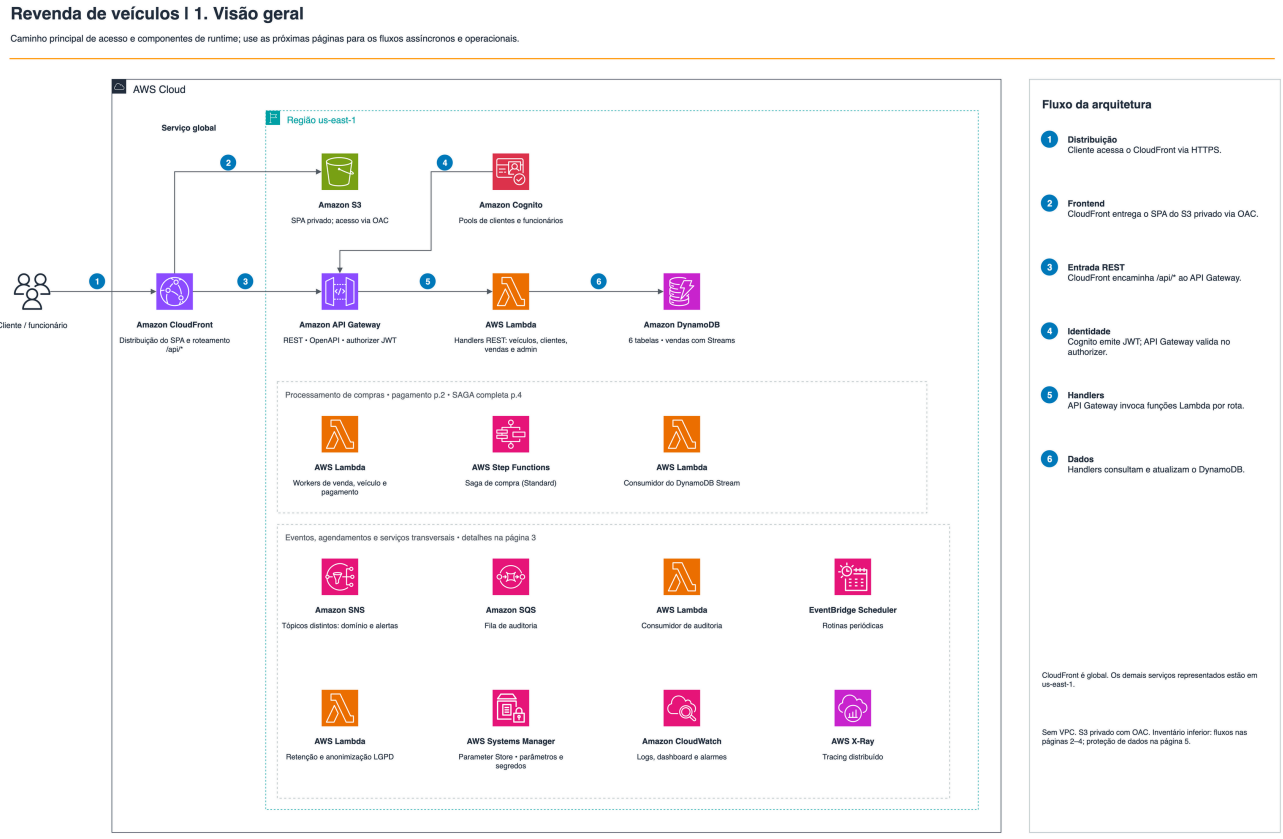
5. Custo e limites operacionais

O desenho prioriza serviços gerenciados e redução de operação, não gratuidade garantida. Franquias, créditos e elegibilidade da conta não foram verificados. DynamoDB provisionado, transições da SAGA e observabilidade podem gerar cobrança; o Budget criado no bootstrap é um alerta, não um limite. O gateway é simulado e a documentação de retirada não integra órgão de trânsito.

6. Convenções dos diagramas

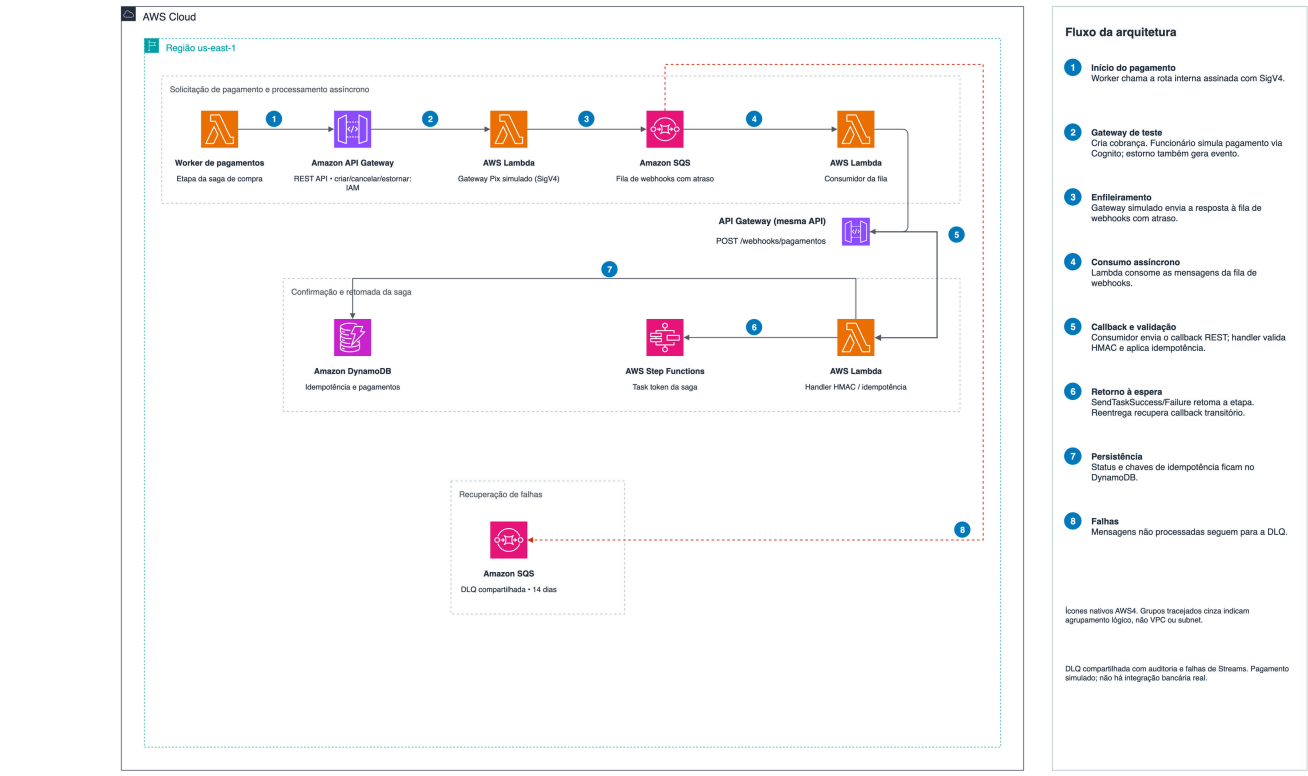
Fonte editável: `docs/diagrams/arquitetura-aws-revenda.drawio`, com ícones AWS4 nativos. Não se representam VPC ou subnets inexistentes. Ícones repetidos do API Gateway representam a mesma API em etapas diferentes.

Diagrama 1 — Visão geral



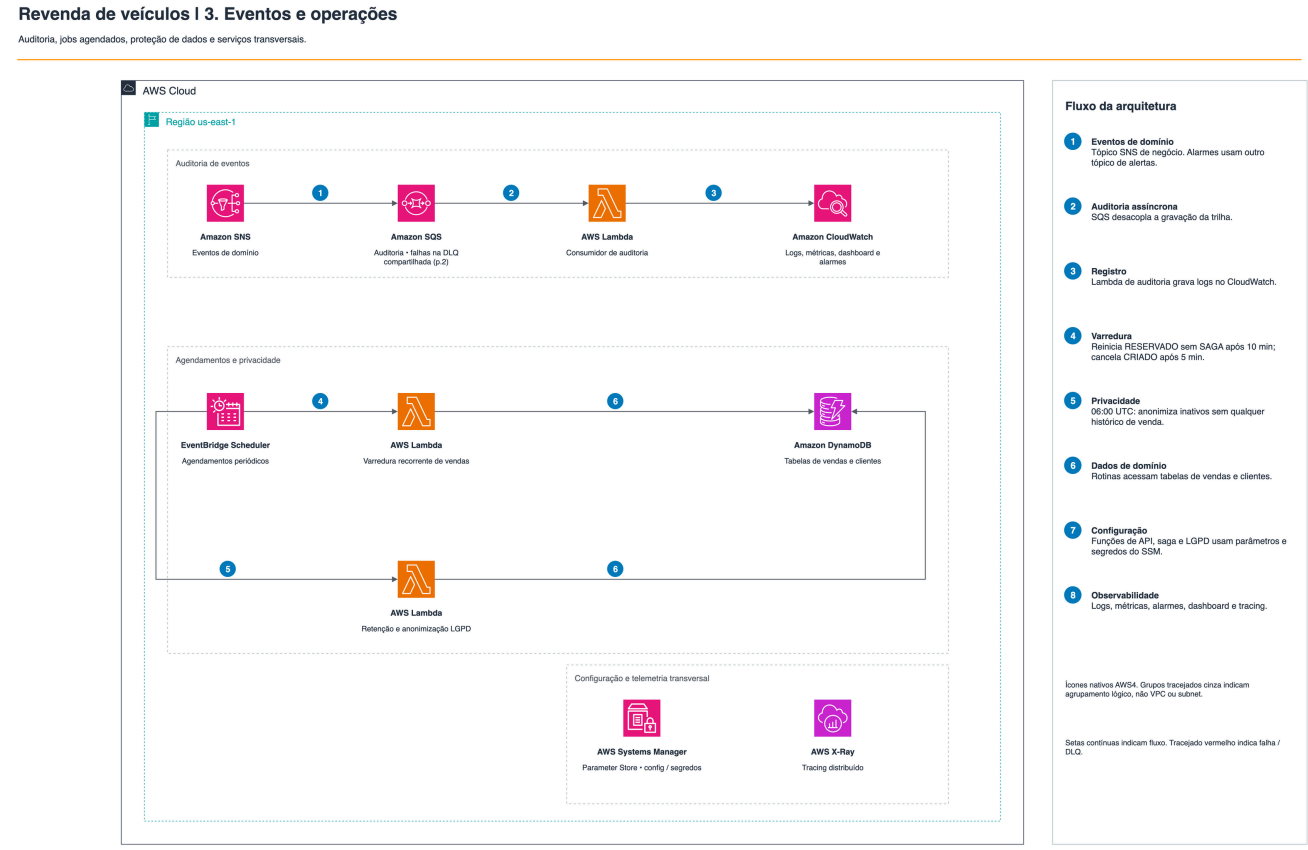
Fonte editável: arquitetura-aws-revenda.drawio, página 1. Serviços declarados; implantação não verificada.

Diagrama 2 — Pagamento e callback



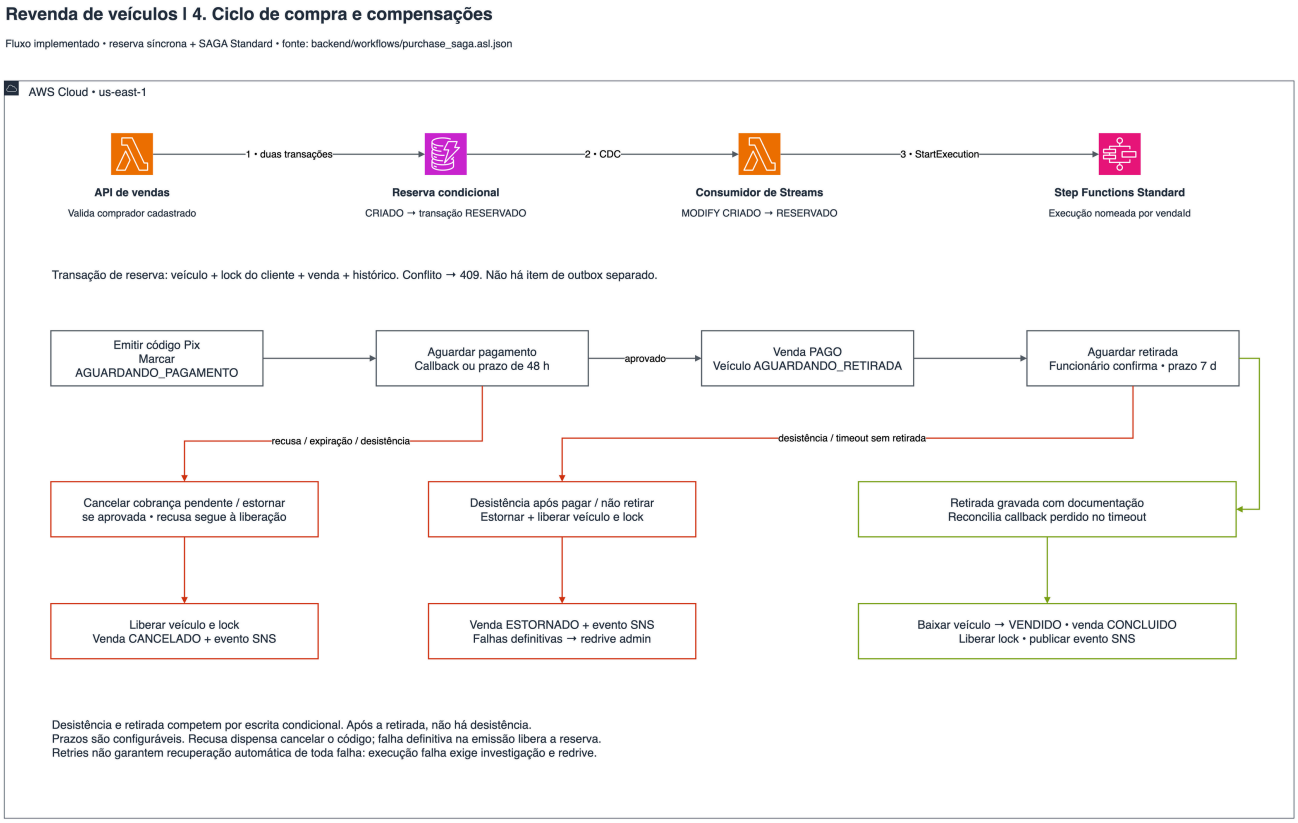
Fonte editável: arquitetura-aws-revenda.drawio, página 2. Serviços declarados; implantação não verificada.

Diagrama 3 — Eventos e operações



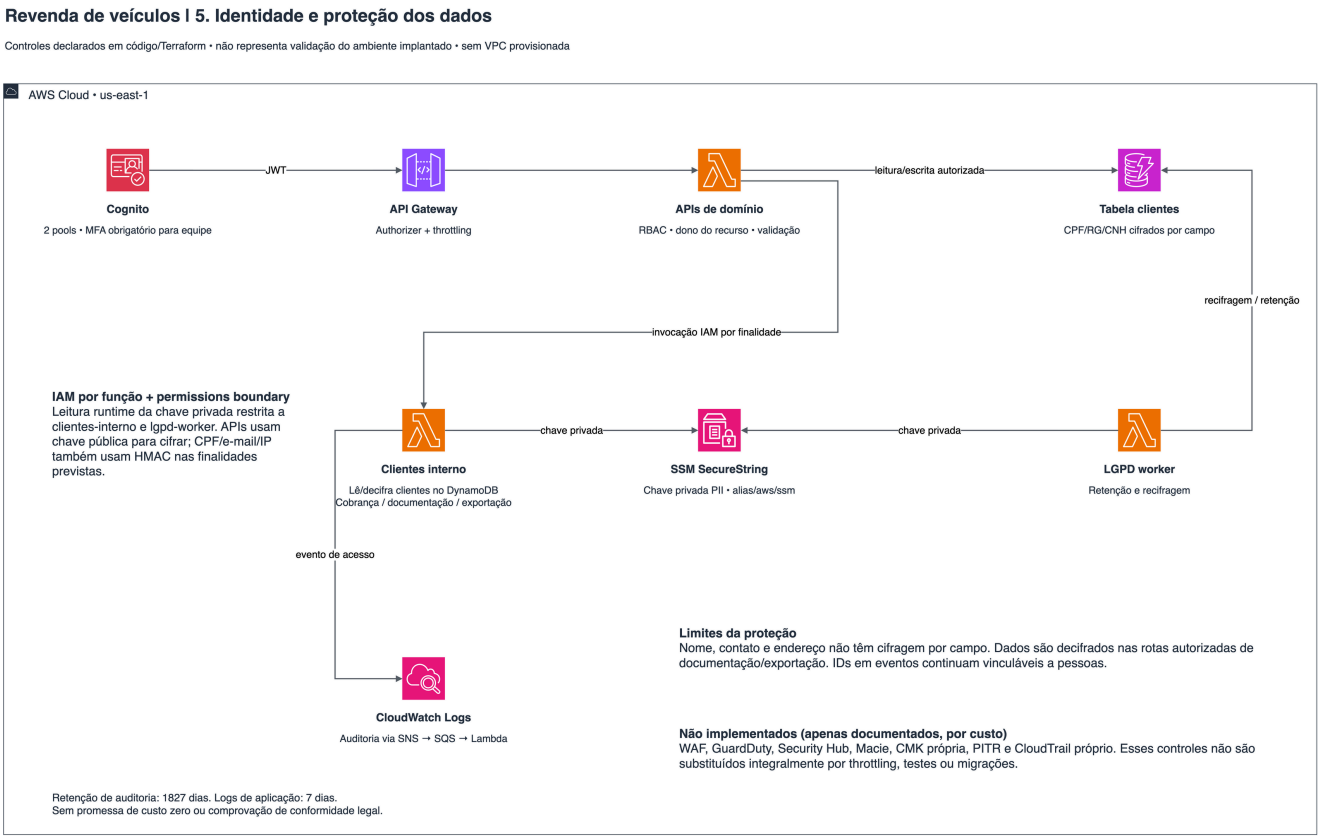
Fonte editável: arquitetura-aws-revenda.drawio, página 3. Serviços declarados; implantação não verificada.

Diagrama 4 — Ciclo completo e compensações



Fonte editável: arquitetura-aws-revenda.drawio, página 4. Serviços declarados; implantação não verificada.

Diagrama 5 — Identidade e proteção de dados



Fonte editável: arquitetura-aws-revenda.drawio, página 5. Serviços declarados; implantação não verificada.